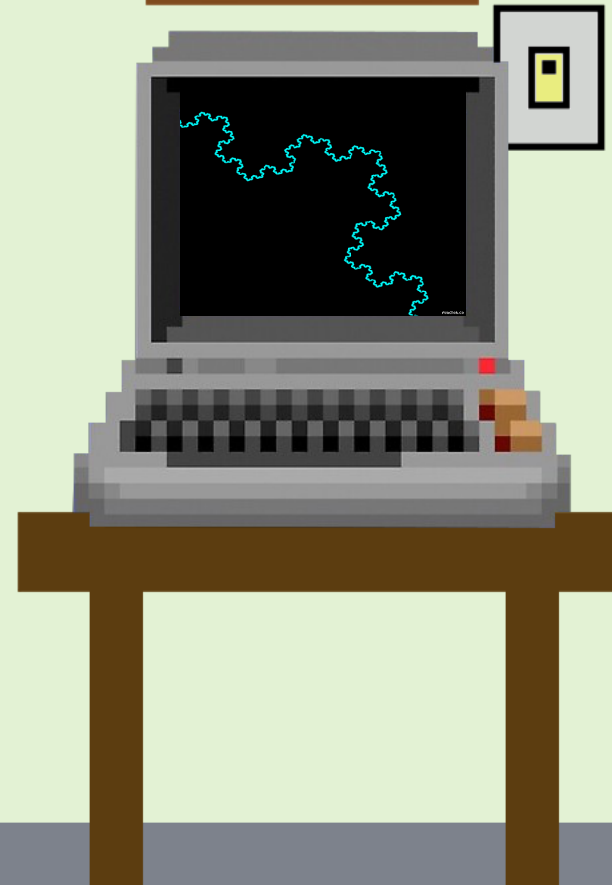


Tipos em Haskell

Programação Funcional



Sistema de Tipos

■ Forte

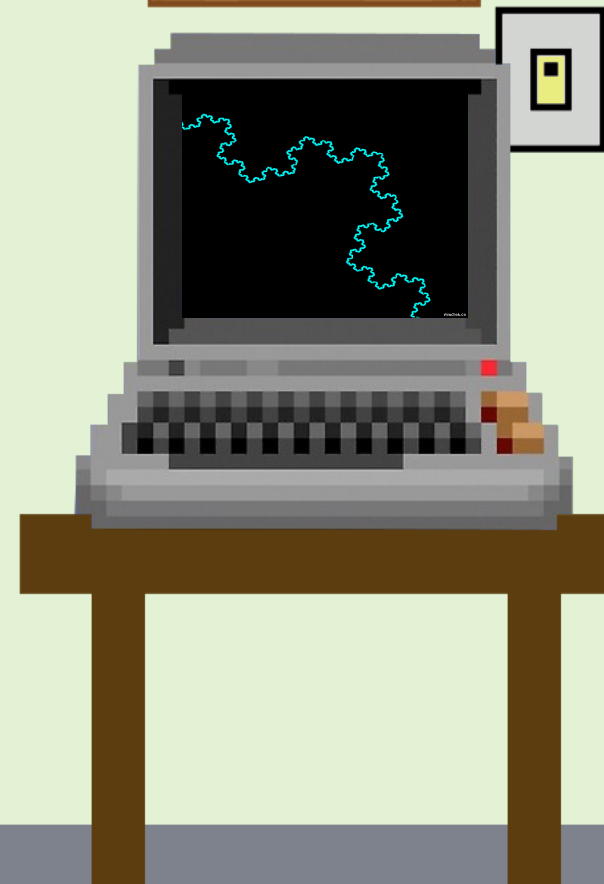
- Compilador verifica tipos das expressões antes de avaliar
- Coerções de valores devem ser explícitas
- Evita bugs antes que eles ocorram

■ Estático

- O tipo de cada valor é conhecido durante a compilação
- É possível identificar erros de tipagem na compilação

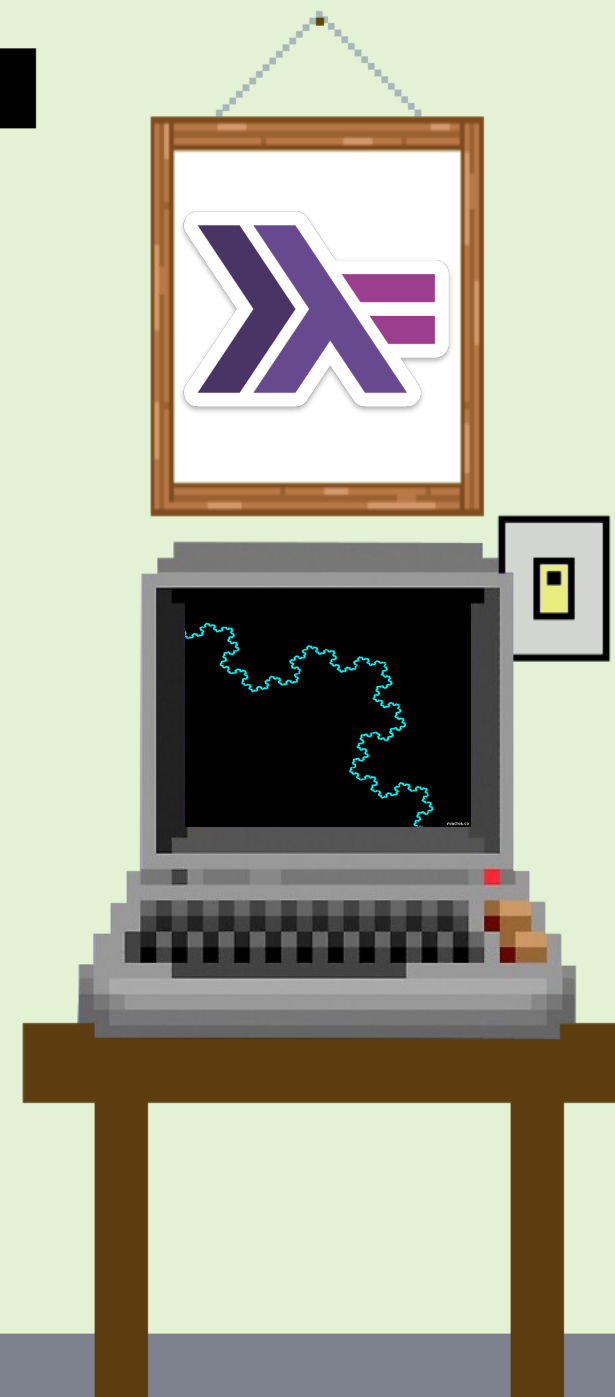
■ Inferido

- Um compilador Haskell pode automaticamente deduzir o tipo de quase todas as expressões em um programa
- Torna a declaração explícita de tipo opcional
- O tipo inferido é o mais amplo possível para a expressão



Tipos Primitivos

- Char - Representa um caractere Unicode
- Bool - Valores da lógica booleana (True e False)
- Int - Representa números inteiros com sinal
 - Depende da arquitetura do sistema (mín de 28 bits)
- Integer - Tamanho depende da memória
 - Menos utilizado por ter um maior custo operacional
- Float - Números de ponto-flutuante
- Double - Números de ponto-flutuante
 - Tipicamente utiliza 64 bits



Operações com Inteiros

Operador	Tipo	Descrição
<code>+</code> , <code>*</code>	<code>Int -> Int -> Int</code>	adição e multiplicação
<code>^</code>	<code>Int -> Int -> Int</code>	exponenciação
<code>-</code>	<code>Int -> Int -> Int</code>	subtração (infixa) e inversor de sinal (prefixa)
<code>div</code>	<code>Int -> Int -> Int</code>	divisão inteira (prefixa), ou 'div' (infixa)
<code>mod</code>	<code>Int -> Int -> Int</code>	módulo (prefixa), ou 'mod' (infixa)
<code>abs</code>	<code>Int -> Int</code>	valor absoluto de um inteiro
<code>negate</code>	<code>Int -> Int</code>	troca o sinal de um inteiro
<code>></code> , <code>>=</code> , <code>==</code>	<code>Int -> Int -> Bool</code>	operadores elacionais
<code>==</code> , <code><=</code> , <code><</code>	<code>Int -> Int -> Bool</code>	operadores relacionais



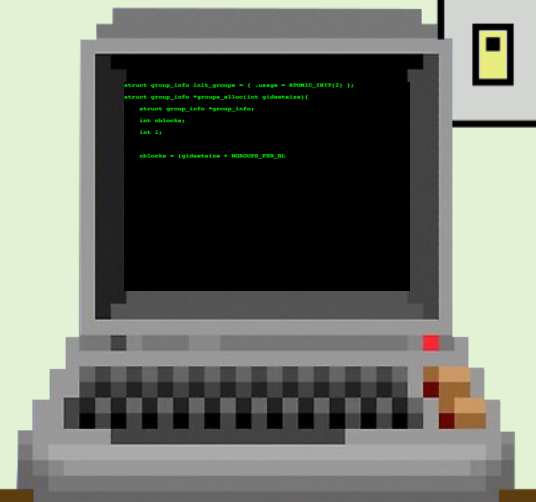
Operações com Inteiros

- Exemplo: Seja uma empresa de vendas que necessita de respostas para as questões a seguir
 - Q1: Qual o total de vendas desde a semana 0 até a semana n ?
 - Q2: Qual a maior venda semanal entre as semanas 0 e n ?
 - Q3: Em que semana ocorreu a maior venda?
 - Q4: Existe alguma semana na qual nada foi vendido?
 - Q5: Em qual semana não houve vendas? (se houve alguma)



Operações com Inteiros

- Considere uma função que retorne a quantidade de vendas em uma dada semana
- Utilizando casamento de padrões
 - `venda::Int->Int`
 - `venda 0 = 5`
 - `venda 1 = 6`
 - `venda 2 = 4`
 - `venda 3 = 8`
 - `venda 4 = 2`
 - `venda 5 = 17`



Operações com Inteiros

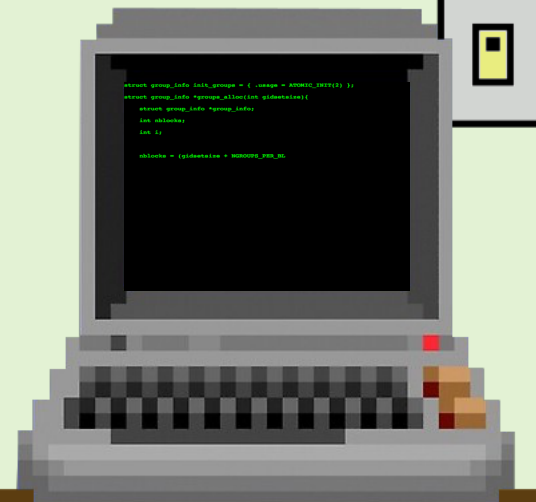
- Q1: Qual o total de vendas desde a semana 0 até a semana n ?

`totaldeVendas :: Int -> Int`

`totaldeVendas n`

`| n == 0 = venda 0`

`| otherwise = totaldeVendas (n-1) + venda n`



Operações com Inteiros

- E se chamarmos totaldeVendas (-2)?

`totaldeVendas :: Int -> Int`

`totaldeVendas n`

`| n == 0 = venda 0`

`| n > 0 = venda n + totaldeVendas (n - 1)`

`| otherwise = 0`



Operações com Inteiros

- Questão 2: Qual a maior venda semanal entre as semanas 0 e n?

`maiorVenda :: Int -> Int`

`maiorVenda n`

`| n == 0 = venda 0`

`| maiorVenda (n-1) > venda n = maiorVenda (n - 1)`

`| otherwise = venda n`



Operações com Inteiros

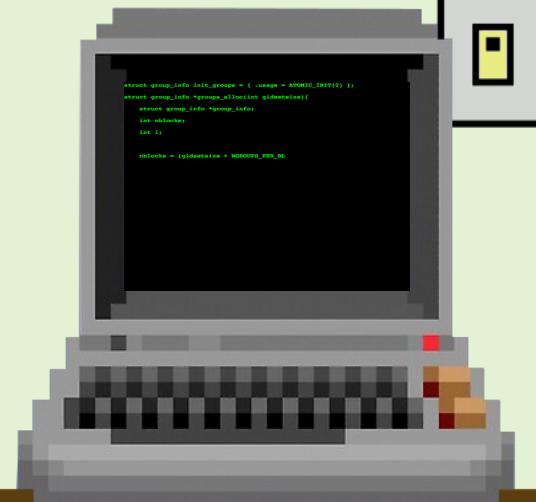
- Refazer utilizando uma função que retorna o maior número entre dois (máximo)

```
maiorVenda :: Int -> Int
```

```
maiorVenda n
```

```
  | n == 0 = venda 0
```

```
  | otherwise = max (maiorVenda (n - 1)) (venda n)
```



Exercício

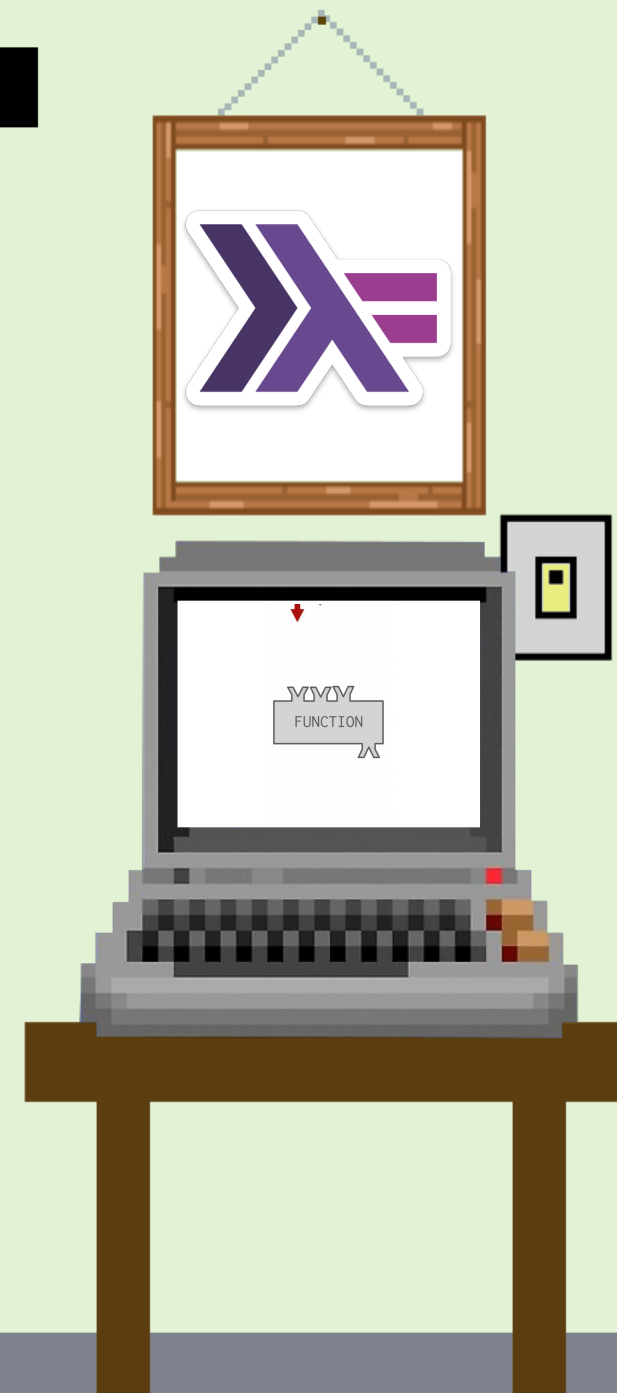
- Em que semana ocorreu a maior venda?
- Existe alguma semana na qual nada foi vendido?
- Em qual semana não houve vendas? (se é que houve alguma)



Tipo Ponto Flutuante

- Tipos mais comuns: Float e Double
- Muitos operadores matemáticos são sobrecarregados para diferentes tipos numéricos (classes de tipos)

Operador	Tipo	Descrição
<code>+, -, *</code>	<code>Float -> Float -> Float</code>	operadores aritméticos
<code>/</code>	<code>Float -> Float -> Float</code>	divisão
<code>^</code>	<code>Int -> Int -> Int</code>	exponenciação
<code>^^</code>	<code>Float -> Int -> Float</code>	exponenciação
<code>**</code>	<code>Float -> Float -> Float</code>	exponenciação
<code>==, / =, <, >, <=, >=</code>	<code>Float -> Float -> Bool</code>	operadores lógicos
<code>abs</code>	<code>Float -> Float</code>	valor absoluto
<code>acos, asin, atan</code>	<code>Float -> Float</code>	funções trigonométricas
<code>ceiling, floor, round</code>	<code>Float -> Float</code>	arredondamentos
<code>cos, sin, tan</code>	<code>Float -> Float</code>	funções trigonométricas



Tipo Ponto Flutuante

Operador	Tipo	Descrição
<i>exp</i>	Float -> Float	exponencial
<i>fromInt</i>	Int -> Float	conversão
<i>log</i>	Float -> Float	logaritmo base 10
<i>logBase</i>	Float -> Float-> Float	logaritmo base n
<i>negate</i>	Float -> Float	inverte sinal
<i>read</i>	String -> Float	converção
<i>pi</i>	Float	valor de pi
<i>show</i>	* -> String	transforma em String
<i>signum</i>	Float -> Int	transforma em Int
<i>sqrt</i>	Float -> Float	raiz quadrada real



Outros Tipos Numéricos

Tipo	Descrição
<i>Double</i>	Ponto flutuante de precisão dupla
<i>Float</i>	Ponto flutuante de precisão simples
<i>Int</i>	Inteiro sinalizado de precisão fixa ($-2^{29} .. 2^{29} - 1$)
<i>Int8</i>	Inteiro sinalizado de 8 bits
<i>Int16</i>	Inteiro sinalizado de 16 bits
<i>Int32</i>	Inteiro sinalizado de 32 bits
<i>Int64</i>	Inteiro sinalizado de 64 bits
<i>Integer</i>	Inteiro sinalizado de precisão arbitrária
<i>Rational</i>	Números racionais de precisão arbitrária
<i>Word</i>	Inteiro não sinalizado de precisão fixa
<i>Word8</i>	Inteiro não sinalizado de 8 bits
<i>Word16</i>	Inteiro não sinalizado de 16 bits
<i>Word32</i>	Inteiro não sinalizado de 32 bits
<i>Word64</i>	Inteiro não sinalizado de 64 bits



Tipo Booleano

- Únicos valores possíveis são True e False
- Funções pré-definidas

Função	Nome	Tipo
&&	and	$\&\& :: Bool \rightarrow Bool \rightarrow Bool$
	or	$:: Bool \rightarrow Bool \rightarrow Bool$
not	inversor	$not :: Bool \rightarrow Bool$



```
account_group_info (info_group = 1, image = MEMO1_00101) {}  
account_group_info "image_allfunction (idatabase){  
  account_group_info "image_info  
  let database  
  let {}  
  database = (idatabase + MEMO1_001_00_00)
```

Exercícios

- Defina uma função `nAnd::Bool->Bool->Bool` que gera como resultado `True`, exceto quando ambos os argumentos são `True`
- Defina uma função `numEqualMax::Int->Int->Int->Int` onde `numEqualMax n m p` retorna a quantidade de números iguais ao máximo entre `n`, `m` e `p`



Tipo Caractere

- São literais escritos entre aspas simples
- Funções pré-definidas para conversão de caracteres em número e vice-versa
 - `toEnum :: Enum a => Int -> a`
 - `fromEnum :: Enum a => a -> Int`
- Existem alguns caracteres que são especiais por terem uso específico:

`'\t'` - tabulação

`'\n'` - nova linha

`'\\'` - uma barra invertida

`'\"'` - aspas simples

`'\"'` - aspas duplas

`'\n'` - caractere ASCII de valor inteiro n



Tipo String

- Duas formas de representar cadeias de caracteres
- Tipo String
 - Utilizando aspas duplas
 - Ex: "string"
- Lista de caracteres
 - Utilizando uma lista
 - Ex: ['s','t','r','i','n','g']
- Todas as funções aplicáveis a listas podem ser utilizadas sobre strings



Funções sobre String

- `putStr :: String -> IO()`
- `show :: t -> String`
- `read :: String -> t`
- Exemplo

```
putStr "IFMA\tCaxias"  
putStr ("IFMA"++ "-" ++ "Caxias")  
show (5+7)  
show (True && False)  
read "True" :: Bool  
read "14" :: Int
```

